Tutorial: C Programming - Debugging Tools

---

1. Fundamentals of Errors and Debugging

Key Definitions

- **Error:** A mistake made by the programmer during the coding stage

  Eg- programmer typing a=b instead of a==b

- **Bug:** Flaw that leads unexpected error in the expected results, the result might not be the one we want it to be
- **Debugging:** The systematic process of finding, analyzing, and removing bugs or defects in a computer program.

---

2. Classifications of Errors in C

Error Type	Description	Primary Cause	Example
Syntax Errors	Code violates the grammatical rules of the C language.	Missing semicolons, unmatched braces, typos in keywords.	int a = 10 (missing ;)
Compilation / Type Errors	Operations performed on incompatible types or missing declarations.	Assigning float string to int pointer, using undeclared variable.	int *p = "hello";
Linker Errors	The compiler generates object files, but the linker cannot combine them.	Function declared but not defined, missing libraries.	undefined reference to 'main'
Runtime Errors	The program crashes or terminates abnormally during execution.	Division by zero, accessing invalid memory addresses.	int x = 10 / 0;
Logical Errors	Code runs to completion without crashing, but output is incorrect.	Wrong mathematical formula, incorrect loop condition.	for(i=0; i<5; i++) instead of <=5

Why Does a Program Compile Successfully but Produce Incorrect Output?

Compiler verifies the syntax of the code that the programmer wrote, its job is to verify, if the rules of the language are being followed or not. Compiler can't tell what the code is trying to do.

Case 1: Syntax & Compilation Errors

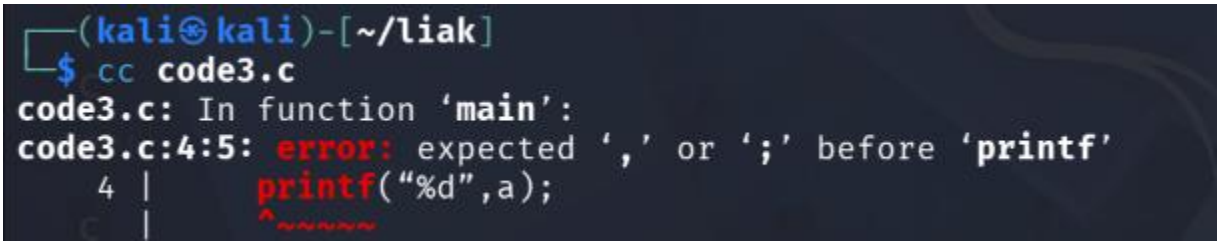
When the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column, and error description.

Example Code (sample1.c)

```
#include <stdio.h>
int main()
{
```

```
    int a = 10
    printf("%d\n", a);
    return 0;
}
```

Compilation Command & Output



Breakdown of Error Message

- **Filename:** code3.c
- **Line & Column:** Line 4, Column 5
- **Type of Message:** error
- **Description:** Expected , or ; before printf statement.

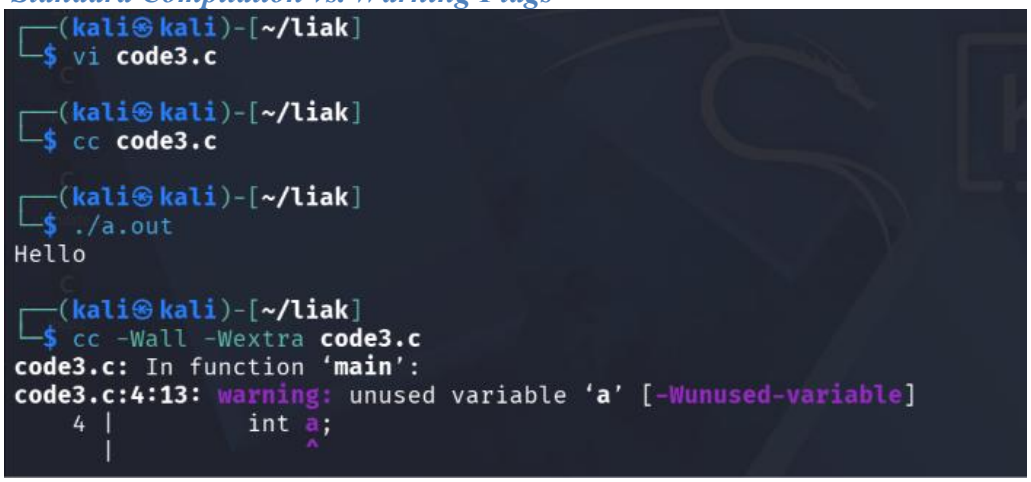
Case 2: Compiler Warnings (-Wall, -Wextra)

A **warning** indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do **not** stop compilation, but ignoring them frequently leads to bugs at runtime.

Example Code (sample2.c)

```
#include <stdio.h>
int main()
{
    int a;
    printf("Hello\n");
    return 0;
}
```

Standard Compilation vs. Warning Flags



- **-Wall**: Enables a broad set of commonly useful compiler warnings.
- **-Wextra**: Enables additional warnings not covered by -Wall.

Case-3: printf-Based Debugging (Logical bug)

Before using complex debugging tools, simple program flow and state inspection can be performed using output statements (printf).

Basic Value Tracing

```
printf("DEBUG: a = %d, b = %d\n", a, b);
```

Example: Tracking Loop Execution (sample3.c)

The following program adds numbers from 1 to 4 instead of 1 to 5 due to a logical bug.
Debug printf() statements help expose the exact iteration behavior.

```
#include <stdio.h>
int main(void)
{
   int i, sum = 0;
   for(i = 0; i < 5; i++)
   {
      printf("DEBUG: entering iteration i = %d\n", i);
      sum = sum + i;
      printf("DEBUG: sum updated to = %d\n", sum);
   }
   printf("Final Sum = %d\n", sum);
   return 0;
}
```

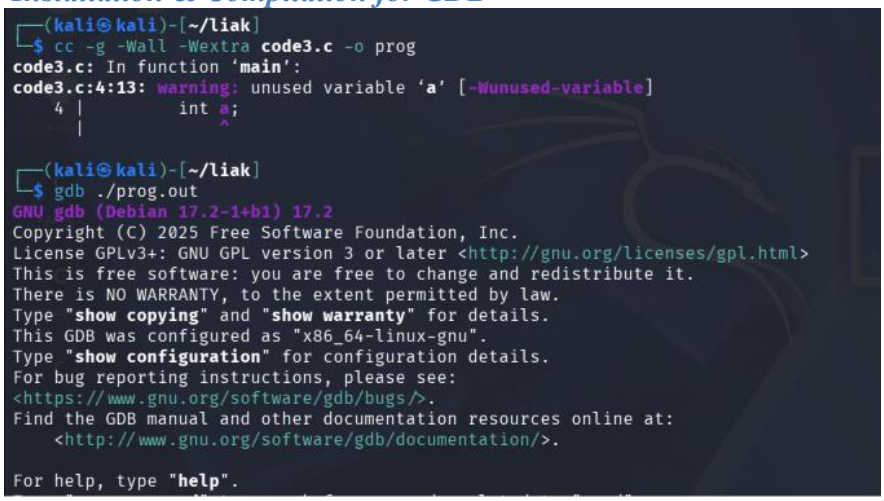
---

3. Introduction to GDB (GNU Debugger)

Background: What is GNU & GDB?

- **GNU:** A recursive acronym for *"GNU's Not Unix!"*. It is an open-source operating system project initiated by Richard Stallman in 1983 to provide a completely free Unix-like system.
- **GDB (GNU Debugger):** The standard debugger for the GNU software system, allowing developers to inspect variable values, set execution breakpoints, and analyze crashes.

Installation & Compilation for GDB



Quick Reference: Common GDB Commands

Command	Shorthand	Description / Purpose
break <line / func>	b	Sets a breakpoint at a specified line number or function name.
run	r	Starts execution of the program from the beginning.
next	n	Executes the next line of code (steps **over** function calls).
step	s	Executes the next line of code (steps **into** function calls).

print <var>	p	Prints the current value of a variable or expression.

display <var>	disp	Automatically prints the value of a variable after every step.
continue	c	Resumes execution until the next breakpoint or program completion.
list	l	Prints source code lines around the current execution position.
info locals	info loc	Lists all local variables and their current values in the current frame.
backtrace	bt	Displays the current call stack (chain of function calls).
info breakpoints	i b	Displays all active breakpoints and their numbers.
delete <id>	d	Deletes the specified breakpoint number (or all if omitted).
disable <id>	dis	Temporarily disables a breakpoint without deleting it.
enable <id>	en	Re-enables a previously disabled breakpoint.
quit	q	Exits the GDB debugging session.

Case 4: Debugging a Logical Error using GDB

Example: Code with Logic Bug (sample4.c)

```
#include <stdio.h>                          // Line 1
int main()                                  // Line 2
{                                           // Line 3
   int i, sum = 0;                          // Line 4
   for(i = 1; i < 5; i++)                   // Line 5 (Logical Bug: should be i <= 5)
   {                                        // Line 6
      sum = sum + i;                        // Line 7
   }                                        // Line 8
   printf("Sum = %d\n", sum);               // Line 9
   return 0;                                // Line 10
}
```

- **Expected Output:** 15 (1 + 2 + 3 + 4 + 5)
- **Actual Output:** 10 (1 + 2 + 3 + 4)

Debugging Walkthrough in GDB

```
For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./a.out ...
(gdb) list
1       #include <stdio.h>
2       int main()
3       {
4               int i, sum = 0;
5               for(i = 1; i < 5; i++)
6               {
7                       sum = sum + i;
8               }
9               printf("Sum = %d\n", sum);
10              return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file code3.c, line 7.
(gdb) run
Starting program: /home/kali/liak/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at code3.c:7
7                       sum = sum + i;
(gdb) print i
$1 = 1
(gdb) print i
$1 = 1
(gdb) dislay i
❌ Undefined command: "dislay".  Try "help".
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at code3.c:7
7                       sum = sum + i;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at code3.c:7
7                       sum = sum + i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at code3.c:7
7                       sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10
[Inferior 1 (process 22995) exited normally]
(gdb) sS
```

Conclusion: The loop terminates when i reaches 5 without executing the loop body for 5. The loop condition should be changed from i < 5 to i <= 5.

Case 5: Debugging Functions (next vs. step)

- **next:** Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.
- **step:** Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

Example Code (sample5.c)

```
#include <stdio.h>
int square(int x)
{
   int result1;
   result1 = x * x;
   return result1;
}
int main()
{
   int result, n = 5;
   result = square(n);
   printf("Square = %d\n", result);
   return 0;
}
```

Option A: Stepping Over with next

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
(gdb) break main
Breakpoint 1 at 0x1156: file code3.c, line 10.
(gdb) run
Starting program: /home/kali/liak/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at code3.c:10
10              int result, n = 5;
(gdb) next
11              result = square(n);
(gdb) step
square (x=5) at code3.c:5
5               result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6               return result1;
(gdb) print result1
$2 = 25
(gdb) s
```

Option B: Stepping Into with step

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

Example Code (sample6.c)

```
#include <stdio.h>
int divide(int a, int b)
{
   return a / b;                    // Division by zero when b = 0
}
int calculate(int x)
{
   return divide(x, 0);             // Passes 0 as divisor
}
int main()
{
   int result = calculate(10);
   printf("Result: %d\n", result);
   return 0;
}
```

GDB Backtrace Execution

Understanding Stack Frames

- **Frame #0:** The function where the crash actually occurred (divide).
- **Frame #1:** The function that called frame #0 (calculate).
- **Frame #2:** The top-level function that started the call sequence (main).

You can inspect specific frames using the frame command:

```
(gdb) frame 0
(gdb) print b          # Shows b = 0, identifying division by zero
(gdb) frame 1
(gdb) print x          # Shows x = 10
```

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behavior or memory corruption.

Example Code (sample7.c)

```
#include <stdio.h>
int main()
{
   int a[5];
   int i;
    for(i = 0; i <= 5; i++)
   {
      scanf("%d", &a[i]);                    //Line 8
   }
   return 0;
}
```

Inspection in GDB

- **Correct Loop Condition:** for(i = 0; i < 5; i++)

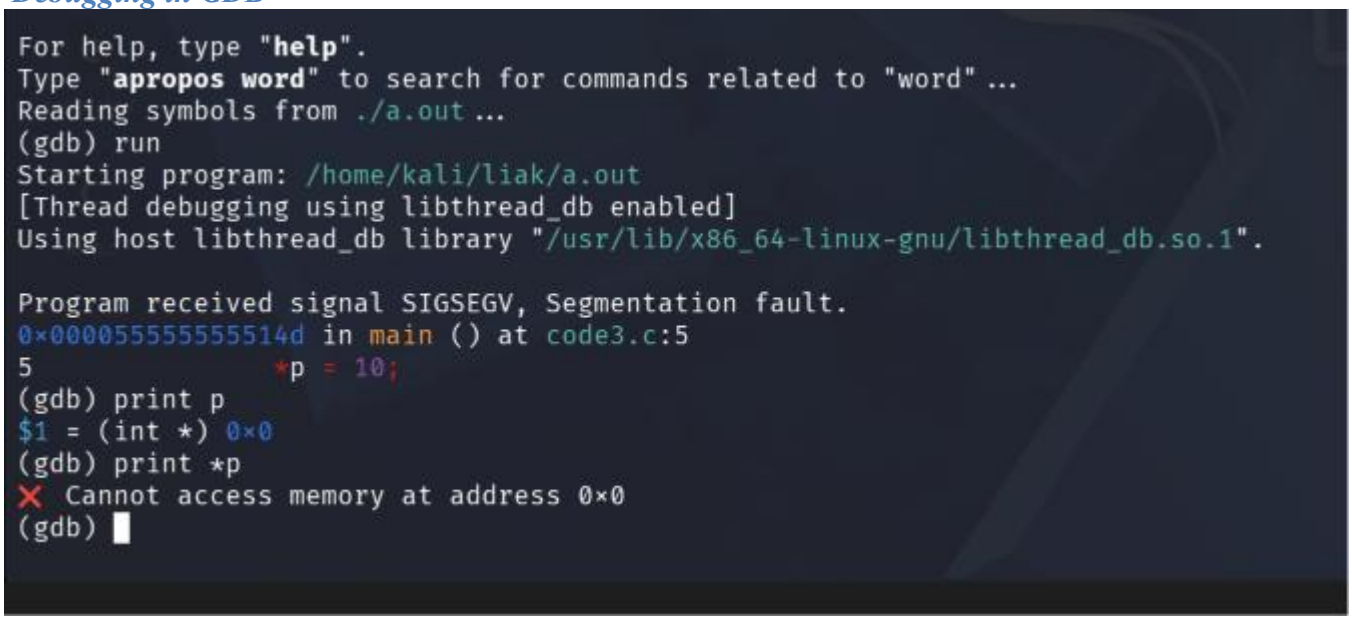
Case 8: Segmentation Faults (SIGSEGV) using GDB

A **Segmentation Fault** occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

Example Code (sample8.c)

```
#include <stdio.h>
int main()
{
   int *p = NULL;              // Pointer points to address 0x0
   *p = 10;                    // Attempting to write to address 0x0 causes Segmentation Fault
   printf("%d\n", *p);
   return 0;
}
```

Debugging in GDB



Corrected Code

```
#include <stdio.h>
int main()
{
   int x;
   int *p = &x;     // Assign p to point to valid memory address of x
   *p = 10;         // Valid write
   printf("%d\n", *p);
   return 0;
}
```

---

4. Systematic Debugging Strategy

5. Exercises & Homework Questions

1. **Error Classification:** Write a small C program snippet illustrating each of the following error types: Syntax error, Compilation/Type error, Linker error, Runtime error, and Logical error.
2. **GNU Concept:** Explain what the GNU project is and how GDB fits into the GNU ecosystem.
3. **Shell Context:** Define what a Shell and Kernel are. Briefly list three common Linux shells (e.g., bash, zsh, fish).
4. Research and contras t the differences between: Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

Answers:

Syntax Error: Occurs when the code violates the grammatical rules of the C language. The compiler cannot parse the code.

Compilation/Type error: Occurs when syntax is correct, but there is a type mismatch or invalid operation that the compiler catches, such as assigning a string to an integer.

Linker Error: Occurs during the linking phase when the compiler successfully compiles the object code, but the linker cannot find the definition for a referenced function or variable.

Runtime Error: Occurs while the compiled program is executing. The code is syntactically and structurally correct but performs an illegal operation (like division by zero or a segmentation fault) that crashes the program.

Logical Error: Occurs when the program compiles and runs without crashing, but produces an incorrect output due to a flaw in the programmer's logic.

2. The GNU Project: Launched by Richard Stallman in 1983, the GNU Project is a mass collaboration effort to develop a completely free, Unix-like operating system. The name is a recursive acronym for "GNU's Not Unix!"

GDB (GNU Debugger) in the Ecosystem: GDB is the standard debugger for the GNU software system. It integrates seamlessly with the GNU Compiler Collection (GCC). When you compile a C program in Kali Linux using GCC with the -g flag, it generates debugging symbols that GDB can interpret.

3. Shell Context: Shell vs. Kernel

- **Kernel:** Kernel helps the user talk to the hardware of the computer, by translating the human inputs into the machine understandable signals.
- **Shell:** A shell is a user-space program that provides a command-line interface (CLI) for users to interact with the kernel.

NAME: TAANISH AGARWAL
BATCH: B-15
SAP ID: 590034662